Fitness Plan 项目开发总结报告 (截至 2025-12-26)

1. 项目概览 (Project Overview)

本项旨在开发一个 Android 健身计划追踪应用，目前已完成 **MVP (最小可行性产品)** 的 UI 构建与基础逻辑，正在进行 **Data Persistence (数据持久化)** 的 Room 框架迁移。

2. 当前项目结构 (Current Project Structure)

```
Fitness_Plan/
├── gradle/libs.versions.toml      # [关键] 依赖版本管理清单
├── app/build.gradle.kts          # [关键] 模块构建脚本 (Kotlin DSL)
├── app/src/main/res/
│   ├── layout/
│   │   ├── activity_main.xml      # 主界面布局 (包含标题和 ListView)
│   │   └── list_item_exercise.xml # 列表项布局 (动作、重量、次数、组数、按钮)
│   └── values/strings.xml         # 字符串资源 (已解耦硬编码文本)
└── app/src/main/java/com/example/fitness_plan/
    ├── MainActivity.java          # [核心] 主逻辑与适配器 (处理多线程与异步更新)
    └── data/ (待完善/新建)         # Room 数据库模块
        ├── ExerciseEntity.java    # 数据库实体类
        ├── WorkoutDao.java        # 数据访问对象接口
        └── AppDatabase.java       # 数据库单例入口
```

3. 已完成的技术优化 (Completed Technical Optimizations)

3.1 UI 与资源规范化 (Lint Optimization)

- **硬编码提取 (Hardcoded Strings)**: 所有的界面文本均已移动至 `strings.xml` ，通过 `@string/` 引用。
- **可访问性 (Accessibility)**: 修复了 `EditText` 的 `autofillHints` 警告，通过 `android:importantForAutofill="no"` 明确意图。

3.2 Java 代码质量提升 (Code Quality)

- **内存泄漏预防 (Memory Leak Prevention)**: 将 `ExerciseAdapter` 改为 `static` 内部类，断开对 Activity 的隐式引用。
- **现代化语法**: 使用 **Lambda 表达式** 替换匿名内部类，代码更简洁。
- **变量约束**: 识别并修复了 `Effectively Final` 错误，确保在 Lambda 内部访问的是稳定的变量镜像。

3.3 构建系统现代化 (Build System)

- **AGP 降级**: 将 Android Gradle Plugin 从不兼容的 `8.13.0` 降级至 IDE 支持的 `8.12.0` 。
- **版本目录 (Version Catalog)**: 成功配置了 `libs.versions.toml` 与 `build.gradle.kts` 的协同工作模式。

4. 正在进行的任务：Room 数据库持久化 (Active Task: Room Migration)

目前正在从“内存存储”切换到“本地数据库存储”，以解决 App 重启后数据丢失的问题。

4.1 技术选型思路 (Decision Logic)

- **方案**: 选用 **Jetpack Room** 框架。
- **优势**: 提供 `Compile-time SQL validation` (编译时 SQL 校验)，减少 `Boilerplate code` (样板代码)。

4.2 核心架构组件 (Architecture Components)

1. `ExerciseEntity`：定义 `exercise_table` 表结构。
2. `WorkoutDao`：抽象增删改查方法，特别是 `getAllExercises()` 和 `update()`。
3. `AppDatabase`：提供线程安全的单例。
4. `Threading` **(多线程)**: 在 `MainActivity` 中引入 `ExecutorService`，确保所有数据库操作在 **Worker Thread** (后台线程) 执行，避免 **ANR** (应用无响应)。

5. 待解决与后续计划 (Next Steps)

1. **同步适配器逻辑**: 将 `ExerciseAdapter` 中的数据源类型从原始 Java 对象彻底改为 `ExerciseEntity`。
2. **实时保存优化**: 完善 `TextWatcher` 逻辑，确保用户修改输入框时，数据能实时 `Auto-save` (自动保存) 到数据库。
3. **UI 细节修复**: 处理已完成动作的 `Strike-through` (删除线) 视觉反馈。

6. 核心术语对照表 (Glossary)

- **Data Persistence (数据持久化)**: 数据不随 App 退出而丢失。
- **ORM (Object-Relational Mapping)**: 对象关系映射。
- **Asynchronous (异步)**: 在后台执行，不干扰前台 UI。
- **Effectively Final (事实上的常量)**: 指声明后未被修改的变量。
- **Dependency Injection (依赖注入概念)**: 在 `toml` 中管理版本，在 `gradle` 中引用。

记录时间: 2025-12-26